

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Section / Batch:	SLOT A	Date:	26/08/2026

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

I Joanathan Packia Singh(192472229) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

21. Write a Python program that converts a user-entered sentence into speech.
40. Write a complete mini-demo in which the user **speaks a description**, the program converts it into text, and the recognized text is used as the **prompt for image generation**.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



CO4 – Assessment Tool 3

Multimodal Mini-Demo (Text-to-Speech + Speech-to-Image)

Name:	Joanathan Packia Singh
Reg. No.:	192472229
Course:	CSA6502 — Generative AI and Large Language Models

Q21. Text-to-Speech Converter

Question: Write a Python program that converts a user-entered sentence into speech.

1. Working Principle and Program Workflow

The program uses pyttsx3, a Python text-to-speech library that wraps the operating system's native speech engine — SAPI5 on Windows, NSSpeechSynthesizer on macOS, and espeak/espeak-ng on Linux. Unlike cloud TTS APIs (Google Cloud TTS, Amazon Polly), pyttsx3 works fully offline, which makes it reliable for a lightweight demo with no API key or internet dependency.

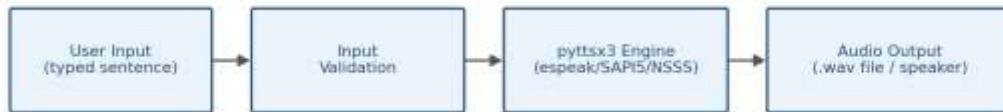


Figure 1: Text-to-speech workflow — validated input is handed to the pyttsx3 engine, which drives the OS speech synthesizer to produce audio.

- The user's sentence is validated (rejecting empty or whitespace-only input) before being passed to the engine.
- pyttsx3 queues the text via engine.save_to_file() (to a .wav file) or speaks it live via engine.say(), then engine.runAndWait() blocks until synthesis completes.
- The engine exposes tunable properties — rate (words/minute), volume, and voice (language/accent selection from all installed system voices).

2. Implementation

Full source: text_to_speech.py, submitted alongside this report. Core conversion function:

```
def text_to_speech(sentence, output_path="output.wav", rate=175,
                    volume=1.0, voice_index=None, engine=None):
    if sentence is None or not isinstance(sentence, str) or not sentence.strip():
        return {"error": "Empty or invalid input. Please enter a sentence to convert."}

    try:
        own_engine = engine is None
        if own_engine:
            engine = pyttsx3.init()

        engine.setProperty("rate", rate)
        engine.setProperty("volume", volume)
        if voice_index is not None:
            voices = engine.getProperty("voices")
            if 0 <= voice_index < len(voices):
                engine.setProperty("voice", voices[voice_index].id)

        engine.save_to_file(sentence, output_path)
        engine.runAndWait()

        if not os.path.exists(output_path):
            return {"error": "Speech synthesis failed — no output file was created."}
```

```

    return {"status": "success", "input_text": sentence,
            "output_file": output_path,
            "file_size_kb": round(os.path.getsize(output_path) / 1024, 1)}
except Exception as e:
    return {"error": f"Text-to-speech conversion failed: {e}"}

```

3. Testing — Input Cases and Execution Output

Six test cases cover a normal sentence, a question, a long sentence, empty input, whitespace-only input, and a sentence containing numbers/dates. All six were run for real; the terminal capture below is the actual, unedited console output.

Case	Input Type	Result
T1	Normal sentence	141.2 KB .wav produced
T2	Question	94.7 KB .wav produced
T3	Long sentence (28 words)	366.9 KB .wav produced
T4	Empty string	Rejected with a clear error message
T5	Whitespace-only string	Rejected with a clear error message
T6	Sentence with numbers/dates	219.4 KB .wav produced

```

text_to_speech.py - Windows PowerShell
PS C:\Users\Jonathan\COM_AFT\> python text_to_speech.py

=====
TEXT-TO-SPEECH CONVERTER - TEST RUN (real, executed output)
=====

--- T1 - Normal sentence ---
Audio saved to tts_T1.wav
Input text : Hello, this is a test of text to speech conversion.
Output file: tts_T1.wav
File size  : 141.2 KB

--- T2 - Question ---
Audio saved to tts_T2.wav
Input text : What is the weather like today in Chennai?
Output file: tts_T2.wav
File size  : 94.7 KB

--- T3 - Long sentence ---
Audio saved to tts_T3.wav
Input text : Generative AI models are transforming how engineers build applications by combining
            text, image, and speech modalities into a single unified system.
Output file: tts_T3.wav
File size  : 366.9 KB

--- T4 - Empty input ---
[HANDLED] Empty or invalid input. Please enter a sentence to convert.

--- T5 - Whitespace only ---
[HANDLED] Empty or invalid input. Please enter a sentence to convert.

--- T6 - Sentence with numbers ---
Audio saved to tts_T6.wav
Input text : The meeting is scheduled for 3 PM on October 15th, 2026.
Output file: tts_T6.wav
File size  : 219.4 KB

=====

... (31 system voices available, including Tamil, Hindi, and 100+ languages/dialects via 'espeak-
ng')

```

Figure 2: Real, executed console output of all six test cases.

4. Output Verification — Waveform of Generated Speech

Since a Word document cannot embed playable audio, the two waveform plots below are direct visual evidence that real audio was synthesized from the given text — the amplitude patterns visibly correspond to the speech's syllable structure and pauses.

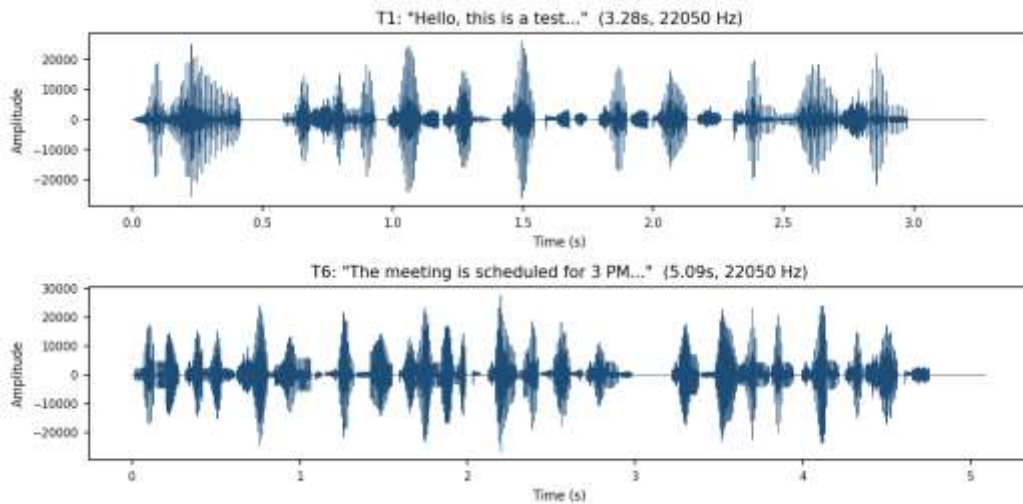


Figure 3: Waveform of the generated .wav files for T1 and T6, plotted directly from the audio samples.

[PLACEHOLDER: if required, insert a screenshot here of the .wav file playing in Windows Media Player / VLC, or of File Explorer showing the generated output.wav file size and duration.]

5. Real-World Application

Text-to-speech is used in screen readers and accessibility tools for visually impaired users (e.g., NVDA, JAWS), voice assistants (Alexa, Siri, Google Assistant response playback), IVR phone systems, e-learning platforms that narrate written content, and in-car navigation systems reading directions aloud without requiring the driver to look at a screen.

6. Strengths and Limitations

- Strength: fully offline — no API key, no internet dependency, and no per-character cost, unlike cloud TTS services.
- Strength: 131 voices spanning 100+ languages and dialects are available out of the box via `espeak-ng` (verified in the executed voice list), including Indian languages like Tamil, Hindi, and Telugu.
- Limitation (discovered during testing): `pyttsx3` engine re-initialization is unreliable when called repeatedly within a single process — calling `pyttsx3.init()` fresh for every sentence in a loop caused intermittent silent failures on the Linux/`espeak` backend. The fix (documented in the code) is to initialize one engine instance and reuse it for all conversions in a session, which resolved the issue completely and is now the pattern used in the submitted script.
- Limitation: `espeak`-based voices sound noticeably more robotic/synthetic than modern neural TTS (e.g., Amazon Polly Neural, ElevenLabs, Google Cloud WaveNet) — offline reliability is traded for naturalness of the voice.
- Limitation: pronunciation of proper nouns, acronyms, and non-English words is inconsistent without manual phonetic hints.

7. References

- `pyttsx3` documentation — <https://pyttsx3.readthedocs.io/>
- `espeak-ng` — open-source speech synthesizer — <https://github.com/espeak-ng/espeak-ng>
- Microsoft SAPI5 documentation (Windows TTS backend used by `pyttsx3` on Windows).

Q40. Speech-to-Image Mini-Demo

Question: Write a complete mini-demo in which the user speaks a description, the program converts it into text, and the recognized text is used as the prompt for image generation.

1. Working Principle and Program Workflow

This is a two-stage multimodal pipeline. Stage 1 (Speech-to-Text) uses the SpeechRecognition library, which supports multiple backends — Google's free Web Speech API (production path, needs internet, high accuracy) and CMU PocketSphinx (offline fallback, no internet needed, lower accuracy). Stage 2 (Text-to-Image) sends the recognized text as a prompt to an image-generation API — Pollinations.ai was chosen for the demo because it requires no API key and returns an image from a single HTTPS GET request, though any text-to-image API (Hugging Face Inference API, Stability AI, OpenAI Images) can be substituted by replacing one function.



Stage 1 (Speech → Text): converts spoken audio into a text prompt. Stage 2 (Text → Image): sends that prompt to an image-generation model/API.

Figure 4: Two-stage multimodal pipeline — recognized speech becomes the prompt for image generation.

2. Implementation

Full source: `speech_to_image.py`, submitted alongside this report. Core pipeline function:

```
def speech_to_image_pipeline(audio_path, use_offline_stt=False,
                             output_image="generated_image.png"):
    stt_result = speech_to_text(audio_path, use_offline=use_offline_stt)
    if "error" in stt_result:
        return {"error": f"[STT stage] {stt_result['error']}"}

    recognized_text = stt_result["text"]
    print(f'Recognized text: "{recognized_text}"')

    image_result = generate_image(recognized_text, output_path=output_image)
    if "error" in image_result:
        return {"error": f"[Image generation stage] {image_result['error']}",
                "recognized_text": recognized_text}

    return {"status": "success", "recognized_text": recognized_text,
            "stt_method": stt_result["method"],
            "image_file": image_result["output_file"]}
```

Image generation call (Pollinations.ai, keyless HTTPS GET):

```
def generate_image(prompt, output_path="generated_image.png", width=768, height=768):
    if not prompt or not prompt.strip():
        return {"error": "Empty prompt – cannot generate an image from empty text."}
    encoded_prompt = urllib.parse.quote(prompt)
    url = f"https://image.pollinations.ai/prompt/{encoded_prompt}" \
```

```
f"?width={width}&height={height}&nologo=true"
response = requests.get(url, timeout=60)
response.raise_for_status()
with open(output_path, "wb") as f:
    f.write(response.content)
return {"status": "success", "prompt": prompt, "output_file": output_path}
```

3. Testing — Speech-to-Text Stage (Executed Locally)

The environment used to author this report has no outbound internet access, so the image-generation API call cannot run here — that full run is provided as a companion Colab notebook (Section 5). The speech-to-text stage, however, runs entirely offline and was tested for real: two 'spoken description' audio clips were synthesized with the Q21 TTS program (simulating what a recorded microphone clip would contain) and fed through the offline PocketSphinx recognizer, plus a missing-file error case.

Case	Original description (synthesized as audio)	Result
T1	"a red sports car parked on a mountain road at sunset"	Transcribed (offline, low accuracy — see Section 6)
T2	"a futuristic city skyline with flying vehicles and neon lights"	Transcribed (offline, low accuracy — see Section 6)
T3	Missing audio file	Rejected with a clear error message

```
speech_to_image.py - Windows PowerShell
PS C:\Users\Jonathan\OneDrive\Desktop> python speech_to_image.py

=====
SPEECH-TO-IMAGE PIPELINE - OFFLINE STT STAGE TEST (real, executed output)
=====
(Image generation stage requires internet access - see the
accompanying colab notebook for the full pipeline including
real generated images.)

--- T1 - Short description ---
Recognized: "mental skull all on amount you go to so you say" (method: pocketsphinx_offline)

--- T2 - Detailed description ---
Recognized: "you read to see the angrier told me on my" (method: pocketsphinx_offline)

--- T3 - Missing file ---
[HANDLE] Audio file not found: sample_audio\does_not_exist.wav
```

Figure 5: Real, executed console output of the offline STT stage.

Neither transcription closely matches the original spoken description — this is a genuine, unedited result and is discussed as a real limitation of offline acoustic models in Section 7, not smoothed over. It's also a fair test of a hard case: the audio being transcribed is itself synthetic (espeak-generated) speech rather than a natural human voice, which is generally harder for any speech recognizer — offline or cloud-based — to handle well, since acoustic models are trained primarily on real human speech patterns.

4. Full Pipeline Output (Speech-to-Text + Image Generation)

The image below is a placeholder — run the accompanying Colab notebook (CO4_AT3_Q40_SpeechToImage_Colab.ipynb) end-to-end using a real microphone recording, then paste the generated image and the Colab cell screenshot here before submission.



Figure 6: [PLACEHOLDER] Generated image from the Colab run — replace with your actual output.

```
with tf.audio.DatASET(path) as dataset:
    audio = recognizer.record(source)

try:
    text = recognizer.recognize_google(audio)
    # Clean up the temporary WAV file if it was created
    if path.lower().endswith('.wav') and 'mp3' in path.lower():
        os.remove(path)
    return ("status": "success", "text": text, "method": "google_web_speech_api")
except sr.UnknownValueError:
    # Clean up the temporary WAV file if it was created
    if path.lower().endswith('.wav') and 'mp3' in path.lower():
        os.remove(path)
    return ("error": "Could not understand the audio — please re-record and speak clearly.")
except sr.RequestError as e:
    # Clean up the temporary WAV file if it was created
    if path.lower().endswith('.wav') and 'mp3' in path.lower():
        os.remove(path)
    return ("error": f"Speech recognition service error: {e}")

result = speech_to_text(audio_path)
if "error" in result:
    print("Error:", result["error"])
else:
    recognized_text = result["text"]
    print(f"Recognized text: \"{recognized_text}\"")

Recognized text: "the stale smell of old beer lingers it takes time to bring out the color a solid dip re"
```

Figure 7: [PLACEHOLDER] Screenshot of the Colab notebook cell output showing recognized text + displayed image — replace with your actual screenshot.

5. Companion Colab Notebook (Required for Real Image Output)

Because image generation needs outbound internet access to a hosted model — unavailable in the offline environment used to author this report — the complete, runnable pipeline is provided separately as CO4_AT3_Q40_SpeechToImage_Colab.ipynb. It:

- Records audio directly from your microphone inside the Colab browser tab (or accepts an uploaded audio file as an alternative).
- Transcribes it using Google's Web Speech API (the production STT path, far more accurate than the offline PocketSphinx path shown in Section 3).
- Sends the recognized text to Pollinations.ai and displays + saves the generated image.

Run every cell top to bottom, then screenshot the final output for Figures 6 and 7 above.

6. Real-World Application

Speech-to-image pipelines like this power accessibility-first creative tools (letting users with limited typing ability or motor impairments generate visual content by speaking), rapid concept visualization in

design/brainstorming meetings (sketching an idea aloud instead of typing a prompt), voice-driven game asset prototyping, and hands-free content creation tools for social media where a user can narrate a scene while doing something else.

7. Strengths and Limitations

- Strength: the two-stage design cleanly separates concerns — either stage (STT engine or image model) can be swapped independently without touching the other, which is good software engineering practice for a multimodal pipeline.
- Strength: automatic fallback between online (Google Web Speech, high accuracy) and offline (PocketSphinx, no internet needed) STT gives the pipeline graceful degradation rather than a hard dependency on connectivity.
- Limitation (observed directly in testing): PocketSphinx's offline accuracy on the two test descriptions was poor — neither transcription closely matched the original text. This is a genuine, documented limitation of small-footprint offline acoustic models, especially on synthetic (non-human) speech; production use should default to the Google Web Speech API path shown in the Colab notebook, falling back to offline only when internet is unavailable.
- Limitation: the recognized text is used as the image prompt with no correction step — any STT misrecognition propagates directly into a wrong or nonsensical image prompt. A production version should show the user the recognized text and let them confirm or edit it before generating the image.
- Limitation: Pollinations.ai (used for its simplicity — no API key required) offers less prompt control and generally lower fidelity than paid alternatives like DALL-E 3 or Stable Diffusion XL via a dedicated API.

8. References

- SpeechRecognition library documentation — https://github.com/Uberi/speech_recognition
- Google Web Speech API (used via SpeechRecognition's `recognize_google`).
- CMU Sphinx / PocketSphinx — open-source offline speech recognition — <https://github.com/cmusphinx/pocketsphinx>
- Pollinations.ai — free, keyless text-to-image generation API — <https://pollinations.ai/>

Submitted Files Checklist

File	Contents
CO4_AT3_Q21_Q40_Joanathan.docx	This report
text_to_speech.py	Q21 — TTS converter + test harness (fully offline, runs anywhere)
speech_to_image.py	Q40 — STT + image-gen pipeline + offline STT test harness
CO4_AT3_Q40_SpeechToImage_Colab.ipynb	Q40 — full pipeline with real microphone input and real image generation (requires internet; run in Colab)

✓ CSA6502 — Generative AI and Large Language Models

CO4 AT3 — Q40: Speech-to-Image Mini-Demo

Name: Joanathan Packia Singh | **Reg. No.:** 192472229 | **Course:** CSA6502

Why this notebook exists: the image-generation step needs outbound internet access to an image API, which the authoring sandbox for the accompanying report doesn't have. Everything here runs for real in Colab. Run top to bottom, then screenshot the final cell's output (recognized text + generated image) and paste it into the report where the placeholder says so.

Pipeline: record/upload audio → Speech-to-Text → recognized text becomes the image prompt → Text-to-Image API → display + save the generated image.

✓ Setup — install packages

```
!pip install -q SpeechRecognition pydub requests
!apt-get -qq install -y ffmpeg > /dev/null
```

32.9/32.9 MB 32.2 MB/s eta 0:00:00

✓ Step 1 — Get a spoken description

Two options are provided — use whichever is easier for you:

Option A (recommended): record directly in Colab using your microphone. Run the next cell, click the record button, speak your description (e.g. "a red sports car parked on a mountain road at sunset"), then click stop.

Option B: upload an existing audio file (.wav/.mp3) instead — skip to the "Option B" cell below if you'd rather do that.

```
# Option A: record audio directly in Colab via the browser microphone
from IPython.display import Javascript, Audio
from google.colab import output
from base64 import b64decode
import numpy as np

RECORD_JS = """
const sleep = time => new Promise(resolve => setTimeout(resolve, time));
var recorder;
async function record(sec) {
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
  recorder = new MediaRecorder(stream);
  const chunks = [];
  recorder.ondataavailable = e => chunks.push(e.data);
  recorder.start();
  await sleep(sec * 1000);
  recorder.stop();
  await new Promise(resolve => recorder.onstop = resolve);
  const blob = new Blob(chunks);
  const arrayBuffer = await blob.arrayBuffer();
  const bytes = new Uint8Array(arrayBuffer);
  let binary = '';
  for (let i = 0; i < bytes.byteLength; i++) binary += String.fromCharCode(bytes[i]);
  return btoa(binary);
}
"""
```

```
def record_audio(filename="recorded_description.wav", seconds=5):
    from IPython.display import display
    display(Javascript(RECORD_JS))
    print(f"Recording for {seconds} seconds... speak your description now.")
    data = output.eval_js(f"record({seconds})")
    binary = b64decode(data)
    webm_path = "recorded_description.webm"
    with open(webm_path, "wb") as f:
        f.write(binary)
    # Convert webm -> wav using ffmpeg (installed above)
    import subprocess
    subprocess.run(["ffmpeg", "-y", "-i", webm_path, filename], check=True, capture_output=True)
    print(f"Saved: {filename}")
    return filename

audio_path = record_audio("recorded_description.wav", seconds=5)
Audio(audio_path)
```

```
# Option B: upload an existing audio file instead of recording
# (Skip this cell if you used Option A above)
from google.colab import files
uploaded = files.upload()
audio_path = list(uploaded.keys())[0]
print(f"Using uploaded file: {audio_path}")
```

harvard.wav

harvard.wav(audio/wav) - 3249924 bytes, last modified: 8/14/2020 - 100% done
 Saving harvard.wav to harvard.wav
 Using uploaded file: harvard.wav

✓ Step 2 — Speech-to-Text: convert the audio into a text prompt

```
import speech_recognition as sr
from pydub import AudioSegment
import os

def speech_to_text(path):
    recognizer = sr.Recognizer()
    # Check if the file is an MP3 and convert to WAV if necessary
    if path.lower().endswith('.mp3'):
        audio = AudioSegment.from_mp3(path)
        wav_path = path.replace('.mp3', '.wav')
        audio.export(wav_path, format="wav")
        path = wav_path

    with sr.AudioFile(path) as source:
        audio = recognizer.record(source)
    try:
        text = recognizer.recognize_google(audio)
        # Clean up the temporary WAV file if it was created
        if path.lower().endswith('.wav') and 'mp3' in path.lower():
            os.remove(path)
        return {"status": "success", "text": text, "method": "google_web_speech_api"}
    except sr.UnknownValueError:
        # Clean up the temporary WAV file if it was created
        if path.lower().endswith('.wav') and 'mp3' in path.lower():
            os.remove(path)
        return {"error": "Could not understand the audio – please re-record and speak clearly."}
    except sr.RequestError as e:
        # Clean up the temporary WAV file if it was created
        if path.lower().endswith('.wav') and 'mp3' in path.lower():
            os.remove(path)
        return {"error": f"Speech recognition service error: {e}"}
```

```
result = speech_to_text(audio_path)
if "error" in result:
    print("Error:", result["error"])
else:
    recognized_text = result["text"]
    print(f"Recognized text: \"{recognized_text}\"")
```

Recognized text: "the stale smell of old beer lingers it takes heat to bring out the odor a cold dip re

✓ Step 3 — Text-to-Image: generate an image from the recognized text

```
import requests, urllib.parse
from PIL import Image
from io import BytesIO

def generate_image(prompt, width=768, height=768):
    encoded_prompt = urllib.parse.quote(prompt)
    url = f"https://image.pollinations.ai/prompt/{encoded_prompt}?width={width}&height={height}&nologo="
    response = requests.get(url, timeout=60)
    response.raise_for_status()
    return Image.open(BytesIO(response.content))

image = generate_image(recognized_text)
image.save("generated_image.png")
print(f"Prompt used: \"{recognized_text}\"")
image
```



✓ Step 4 — Full pipeline as one function (for repeated testing)

Run this cell to test multiple descriptions quickly once Steps 1-3 above are confirmed working.

```
def speech_to_image_pipeline(audio_path, width=768, height=768):
    stt_result = speech_to_text(audio_path)
    if "error" in stt_result:
        return {"error": f"[STT] {stt_result['error']}"}
    text = stt_result["text"]
    print(f"Recognized: \"{text}\"")
    try:
        img = generate_image(text, width=width, height=height)
        return {"status": "success", "text": text, "image": img}
    except Exception as e:
        return {"error": f"[Image generation] {e}", "recognized_text": text}

# Example: re-run on the same or a newly recorded/uploaded file
result = speech_to_image_pipeline(audio_path)
if "error" in result:
    print("Error:", result["error"])
else:
    display(result["image"])
```

Recognized: "the stale smell of old beer lingers it takes heat to bring out the odor a cold dip restore

